

LAPORAN PRAKTIKUM 3
PEMROGRAMAN BERORIENTASI OBJEK
ENKAPSULASI DAN ACCESS MODIFIER



Disusun Oleh:

Nama	: Mandanta Guru Singa
NIM	: 124140147
Mata Kuliah	: Pemrograman Berorientasi Objek
Dosen Pengampu	: Rahman Indra Kesuma, S. Kom., M. Cs.
Kelas	: RB

PROGRAM STUDI TEKNIK INFORMATIKA
INSTITUT TEKNOLOGI SUMATERA

2026

1. Tujuan Praktikum

- Menjelaskan pengertian enkapsulasi.
- Menjelaskan fungsi access modifier dalam Java.
- Membedakan public, private, dan protected.
- Menggunakan getter dan setter.
- Membuat program Java yang menerapkan enkapsulasi.
- Melakukan validasi sederhana menggunakan setter.

2. Alat dan Bahan

- Laptop
- VS Code
- JDK versi 25
- Modul Praktikum PBO 2

3. Langkah Percobaan

3.1. Percobaan 1

- Membuat class Mahasiswa dengan atribut nama, nim, dan prodi(private) disertai setter dan getter nya.



```
1  class Mahasiswa {
2      private String nama, nim, prodi;
3
4      Mahasiswa(String nama, String nim, String prodi) {
5          this.nama = nama;
6          this.nim = nim;
7          this.prodi = prodi;
8      }
9
10     Mahasiswa(String nama, String nim, String prodi, int semester) {
11         this.nama = nama;
12         this.nim = nim;
13         this.prodi = prodi;
14         setSemester(semester);
15     }
16     public void setNama(String nama) {this.nama = nama;}
17     public void setNim(String nim) {this.nim = nim;}
18     public void setProdi(String prodi) {this.prodi = prodi;}
19
20     public String getNama() {return this.nama;}
21     public String getNim() {return this.nim;}
22     public String getProdi() {return this.prodi;}
23 }
```

3.2. Percobaan 2

- Membuat program untuk menampilkan data mahasiswa menggunakan method.

```
1 void tampilkanInfo() {
2     System.out.println("Nama: " + this.nama);
3     System.out.println("NIM: " + this.nim);
4     System.out.println("Prodi: " + this.prodi);
5     System.out.println("Semester: " + this.semester);
6 }
```

3.3. Percobaan 3

- Menambah atribut semester disertai validasi input datanya yang tidak boleh lebih kecil dari 1(semester < 1).

```
1 private int semester;
2 public void setSemester(int semester) {
3     if (semester < 1) {
4         System.out.println("Semester tidak valid!");
5     } else {
6         this.semester = semester;
7     }
8 }
9 public int getSemester() {return this.semester;}
```

3.4. Percobaan 4

- Membuat class MataKuliah dengan atribut private dan method getter setter.



```
1 class MataKuliah {
2     private String kodeMK;
3     private String namaMK;
4     private int sks;
5
6     public MataKuliah(String kodeMK, String namaMK, int sks) {
7         this.kodeMK = kodeMK;
8         this.namaMK = namaMK;
9         this.sks = sks;
10    }
11
12    public String getKodeMK() { return kodeMK; }
13    public void setKodeMK(String kodeMK) { this.kodeMK = kodeMK; }
14
15    public String getNamaMK() { return namaMK; }
16    public void setNamaMK(String namaMK) { this.namaMK = namaMK; }
17
18    public int getSks() { return sks; }
19    public void setSks(int sks) { this.sks = sks; }
20 }
```

3.5. Percobaan 5

- Membuat program lengkap data mahasiswa ITERA berjumlah 3 object.

```

1  class Mahasiswa {
2      private String nama, nim, prodi;
3      private int semester = 1;
4
5      public void setSemester(int semester) {
6          if (semester < 1) {
7              System.out.println("Semester tidak valid!");
8          } else {
9              this.semester = semester;
10         }
11     }
12     public int getSemester() {return this.semester;}
13     Mahasiswa(String nama, String nim, String prodi) {
14         this.nama = nama;
15         this.nim = nim;
16         this.prodi = prodi;
17     }
18
19     Mahasiswa(String nama, String nim, String prodi, int semester) {
20         this.nama = nama;
21         this.nim = nim;
22         this.prodi = prodi;
23         setSemester(semester);
24     }
25     public void setName(String nama) {this.nama = nama;}
26     public void setNim(String nim) {this.nim = nim;}
27     public void setProdi(String prodi) {this.prodi = prodi;}
28
29     public String getName() {return this.nama;}
30     public String getNim() {return this.nim;}
31     public String getProdi() {return this.prodi;}
32
33     void tampilkanInfo() {
34         System.out.println("Nama: " + this.nama);
35         System.out.println("NIM: " + this.nim);
36         System.out.println("Prodi: " + this.prodi);
37         System.out.println("Semester: " + this.semester);
38     }
39
40
41 }
42
43
44 class MataKuliah {
45     private String kodeMK;
46     private String namaMK;
47     private int sks;
48
49     MataKuliah(String kodeMK, String namaMK, int sks) {
50         this.kodeMK = kodeMK;
51         this.namaMK = namaMK;
52         this.sks = sks;
53     }
54
55     public String getKodeMK() { return kodeMK; }
56     public void setKodeMK(String kodeMK) { this.kodeMK = kodeMK; }
57
58     public String getNameMK() { return namaMK; }
59     public void setNameMK(String namaMK) { this.namaMK = namaMK; }
60
61     public int getSks() { return sks; }
62     public void setSks(int sks) { this.sks = sks; }
63 }
64
65 public class Main {
66     public static void main(String[] args) {
67         Mahasiswa mhs1 = new Mahasiswa("Mandanta", "121140147", "Teknik Informatika", 4);
68         Mahasiswa mhs2 = new Mahasiswa("Budi Utomo", "121140001", "Teknik Elektro", 2);
69         Mahasiswa mhs3 = new Mahasiswa("Siti Aminah", "121140099", "Sains Data", 0);
70
71         mhs1.tampilkanInfo();
72         mhs2.tampilkanInfo();
73         mhs3.tampilkanInfo();
74
75         MataKuliah mk1 = new MataKuliah("IF2123", "Struktur Data", 3);
76         System.out.println("Mata Kuliah: " + mk1.getNameMK() + " (" + mk1.getSks() + " SKS)");
77     }
78 }

```

4. Source Code

<https://github.com/DanamiStartrail/Tugas-2-Konsep-Dasar-PBO>

5. Hasil Output Program

```
Semester tidak valid!  
Informasi Mahasiswa:  
Nama: Mandanta  
NIM: 121140147  
Prodi: Teknik Informatika  
Semester: 4  
Nama: Budi Utomo  
NIM: 121140001  
Prodi: Teknik Elektro  
Semester: 2  
Nama: Siti Aminah  
NIM: 121140099  
Prodi: Sains Data  
Semester: 1  
Mata Kuliah: Struktur Data (3 SKS)
```

6. Analisis Program

Program java ini terdiri dari tiga *class* Mahasiswa, MataKuliah, dan Main. Semua atribut pada kelas Mahasiswa (nama, nim, prodi, semester) dan MataKuliah (kodeMK, namaMK, sks) dideklarasikan sebagai *private*. Ini bertujuan untuk melindungi data agar tidak bisa diakses langsung dari luar kelas. Akses data dilakukan melalui *method* **Getter** dan **Setter**.

Pada kelas Mahasiswa, terdapat dua buah *constructor*:

- **Constructor 1:** Menerima parameter nama, nim, dan prodi.
- **Constructor 2:** Menerima parameter yang sama ditambah dengan semester. Ini merupakan contoh penerapan **Constructor Overloading**, yang memberikan fleksibilitas saat pembuatan objek, baik dengan data semester maupun tanpa data semester di awal.

Program memiliki logika validasi pada *method* setSemester. Method ini memastikan bahwa nilai semester tidak boleh kurang dari 1. Jika pengguna memasukkan nilai di bawah 1 (seperti angka 0 pada objek mhs3), program akan memberikan output peringatan "Semester tidak valid!".

Pada method `setSemester` terdapat logika validasi yang bertujuan untuk memastikan nilai semester tidak boleh kurang dari 1. Jika pengguna memasukkan nilai di bawah 1 (seperti angka 0 pada objek `mhs3`), program akan memberikan output peringatan “Semester tidak valid!” dan value dari atribut semesternya akan bernilai 1 (nilai default).

Pada bagian method `main`, program melakukan instansiasi objek yang terdiri dari tiga objek Mahasiswa (`mhs1` – `mhs3`) dengan data yang berbeda-beda untuk mempresentasikan identitas mahasiswa, dan satu objeknya lagi dari kelas `MataKuliah` (`mk1`) untuk mempresentasikan informasi akademik

Method `tampilkanInfo()` pada kelas Mahasiswa berfungsi sebagai pemusat informasi untuk menampilkan seluruh atribut objek ke konsol. Program menggunakan operator penggabungan string (+) untuk memformat tampilan data agar mudah dibaca.

7. Pertanyaan Diskusi

7.1. Apa itu enkapsulasi?

Enkapsulasi adalah konsep PBO untuk membungkus data (atribut) dan metode (perilaku) menjadi satu kesatuan dalam sebuah kelas, serta membatasi akses langsung dari luar kelas. Tujuannya adalah untuk melindungi integritas data.

7.2. Mengapa atribut sebaiknya dibuat private?

Atribut dibuat `private` agar tidak bisa diubah secara sembarangan oleh kelas lain. Ini mencegah terjadinya perubahan data yang tidak diinginkan atau tidak valid secara langsung di memori.

7.3. Apa fungsi getter dan setter?

Getter: untuk mendapatkan value dari sebuah atribut yang bersifat `private`.

Setter: untuk mengubah nilai (serta melakukan sebuah validasi data) value dari sebuah atribut yang bersifat `private`.

7.4. Apa keuntungan enkapsulasi dalam program?

- **Keamanan Data:** Data terlindungi dari akses ilegal.
- **Fleksibilitas:** Kamu bisa mengubah struktur internal kelas tanpa merusak bagian program lain yang menggunakan kelas tersebut.
- **Kemudahan Maintenance:** Kode lebih terorganisir karena aturan mengenai data berada di satu tempat.
- **Kontrol:** Kamu bisa menentukan apakah suatu atribut hanya bisa dibaca saja (*read-only*) atau bisa diubah juga.

7.5. Bagaimana setter bisa digunakan untuk validasi?

Setter memungkinkan kita menyisipkan logika percabangan (if-else) sebelum nilai benar-benar dimasukkan ke dalam atribut. Jika nilai tidak memenuhi syarat, setter bisa menolak perubahan tersebut. Misalnya pada method `setSemester(int semester)` tanpa enkapsulasi (jika semester bersifat public), orang bisa saja menulis `mhs1.semester = -5`; dan program akan menerimanya begitu saja tanpa ada pengecekan.

8. Kesimpulan

Berdasarkan hasil analisis program dan diskusi mengenai konsep Enkapsulasi, dapat disimpulkan bahwa:

1. **Enkapsulasi sebagai Proteksi Data:** Penggunaan akses modifier private pada atribut kelas Mahasiswa dan MataKuliah bukan sekadar aturan penulisan, melainkan langkah keamanan untuk memastikan bahwa data tidak dapat dimanipulasi secara ilegal dari luar kelas. Hal ini menjaga integritas data mahasiswa agar tetap konsisten.
2. **Peran Krusial Setter dalam Validasi:** Melalui metode setter, program tidak hanya bertugas mengubah nilai, tetapi juga berperan sebagai "filter" atau validasi. Contohnya pada logika `setSemester` yang berhasil mencegah input angka 0, sehingga kesalahan input dapat ditangani sebelum merusak logika bisnis program.
3. **Fleksibilitas melalui Overloading:** Penerapan *Constructor Overloading* memberikan efisiensi dalam pembuatan objek. Program mampu menangani skenario data yang beragam (mahasiswa dengan data semester lengkap maupun tidak) tanpa harus membuat banyak kelas yang berbeda.
4. **Keterhubungan dengan Materi Lanjutan:** Pemahaman yang kuat mengenai Enkapsulasi, Getter-Setter, dan Constructor menjadi fondasi utama sebelum melangkah ke materi *Abstract Class* dan *Interface*. Tanpa kontrol data yang baik (Enkapsulasi), implementasi "kontrak" pada *Interface* atau kerangka pada *Abstract Class* akan sulit dikelola dalam sistem yang lebih kompleks.

Secara keseluruhan, program ini telah mendemonstrasikan bagaimana konsep PBO digunakan untuk menciptakan kode yang lebih aman, terorganisir, dan mudah untuk dikembangkan di masa mendatang.